PKCERT AI & Software Development Internship
Task 17: Neural Network Fundamentals

Abdullah Amir

AI and Software Development
27 July 2026

This report covers the theory, derivations and results. The full code and outputs live in the notebook `neural_network_fundamentals.ipynb`. Every core computation – the perceptron's learning rule, the five activation functions and their derivatives, and the 2-layer network's forward and backward passes – is implemented in plain NumPy; no autograd framework computes a gradient anywhere in this submission. scikit-learn appears only for the permitted `MLPClassifier` baseline and for ordinary preprocessing/evaluation utilities.

Part A: Perceptron Fundamentals

A single-layer perceptron with a hard step activation, trained with the classic rule $w \leftarrow w + \eta(y - \hat{y})x$, updated one misclassified sample at a time.

A linearly separable case. Setosa vs Versicolor from Iris, using petal length and width. The perceptron converged in **2 epochs** (2 errors in epoch 1, 0 in epoch 2), reaching 100% training accuracy.

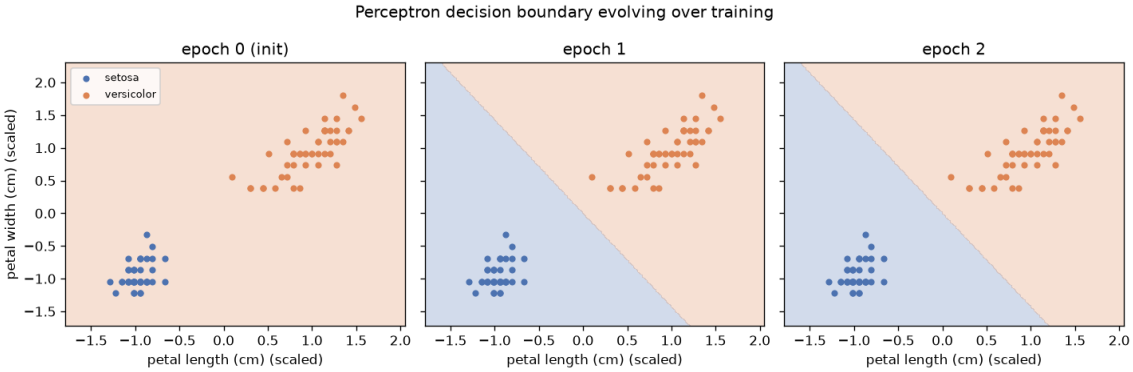

Figure 1: The decision boundary at initialisation, after 1 epoch, and after 2 (converged).

A non-linearly separable case: XOR. The identical algorithm, given XOR's 4 points, ran the full 50 epochs without ever reaching zero errors, plateauing at 4/4 misclassifications and 50% training accuracy – chance level.

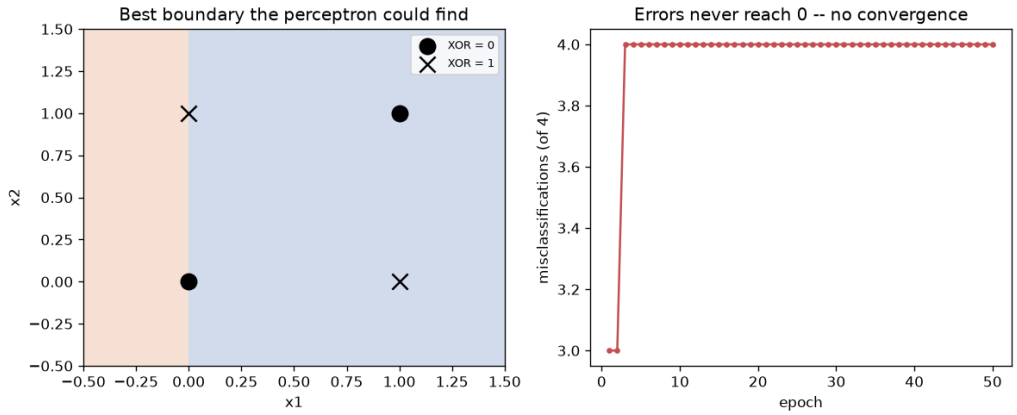

Figure 2: Left: the best single straight-line boundary the perceptron could find. Right: the error count never falls, unlike the Iris case above.

Why XOR defeats a single perceptron. A perceptron's decision boundary is one hyperplane, $w \cdot x + b = 0$. XOR's four points require two separate diagonal boundaries – $(0,0)$ and $(1,1)$ on one side, $(0,1)$ and $(1,0)$ on the other – which is not a linearly separable arrangement. No choice of w, b places a single straight line between the two classes, exactly what the flat, non-decreasing error curve shows: the rule keeps correcting one misclassified point only to break another.

Perceptron vs logistic regression. Both are linear classifiers computing the identical score $z = w \cdot x + b$ and both learn by adjusting w, b from labelled examples; they differ in what happens to z and how that drives learning. The perceptron applies a hard step (predict 1 if $z \geq 0$) and updates weights only on misclassified points, by an amount proportional to the raw error $(y - \hat{y}) \in \{-1, 0, 1\}$ – a mistake-driven rule with no smooth loss surface. Logistic regression applies the smooth sigmoid to z to obtain a probability, defines cross-entropy loss over it, and updates every point's weights by gradient descent on that loss, whose gradient takes the strikingly similar form $(y - p)x$ – except p is a continuous probability, not a hard vote. As the sigmoid's steepness goes to infinity, logistic regression's smooth boundary collapses into the perceptron's hard one: the perceptron is, informally, the hard-threshold, mistake-driven special case of the same linear model that logistic regression fits with a smooth, probabilistic loss.

Part B: Activation Functions

Sigmoid, Tanh, ReLU and Leaky ReLU, plotted with their derivatives over $[-10, 10]$.

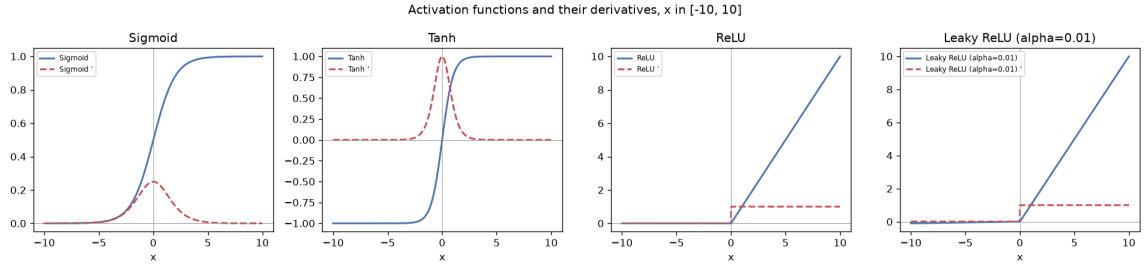

Figure 3: Each function (solid) and its derivative (dashed).

Softmax is not elementwise – every output depends on every logit, and its true derivative is a Jacobian, not a single curve – so it is shown instead by sweeping one logit while the other two

are held fixed.

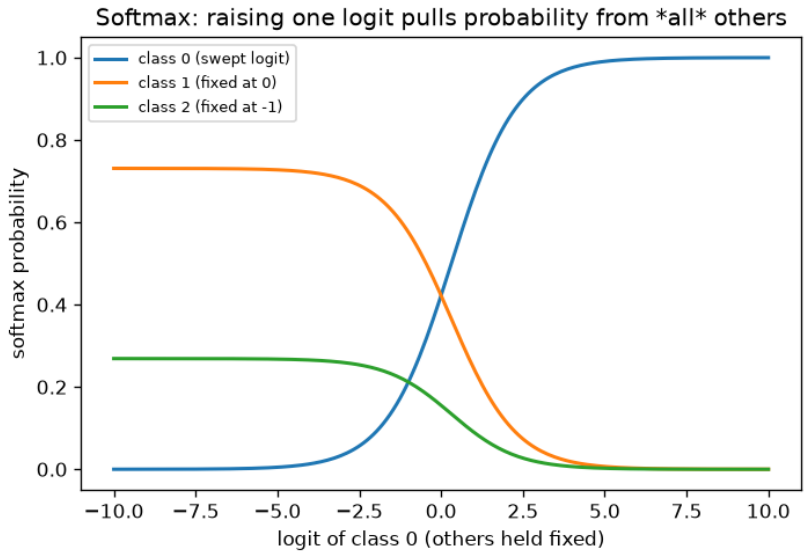

Figure 4: Raising one logit pulls probability mass from *every* other class, not just proportionally – the class fixed at logit 0 (orange) loses more probability than the class fixed at -1 (green) as class 0's logit rises.

Vanishing gradient. Backpropagation multiplies the upstream gradient by each layer's activation derivative (chain rule), so a deep stack multiplies many such derivatives together. Sigmoid's derivative peaks at only **0.25** (at $x = 0$) and is already below 0.01 by $|x| \approx 4.6$; tanh peaks higher, at **1.0**, but still decays toward 0 beyond $|x| \approx 3$. Across the full $[-10, 10]$ range tested, the derivative sits below 10^{-3} for **31.0%** of sigmoid's domain and **58.6%** of tanh's. Stacking several sigmoid or tanh layers multiplies together numbers mostly well below 1, and the product shrinks toward 0 exponentially with depth – the vanishing gradient problem. Sigmoid is the more susceptible of the two, both because its peak derivative is four times smaller and because its outputs are never centred on 0. ReLU's derivative is exactly 1 for any $x > 0$, passing gradient through unchanged no matter how deep the stack – the main reason it replaced sigmoid/tanh as the default hidden-layer choice in deep networks.

Dying ReLU vs Leaky ReLU. ReLU's derivative is exactly 0 for every $x < 0$. If a neuron's weighted input lands below 0 for every training example, every future gradient through it is also exactly 0: its weights stop updating and it is permanently dead. Leaky ReLU fixes this with one change – scaling negative inputs by a small constant $\alpha = 0.01$ instead of clamping to 0 – so the derivative on the negative side is α, not 0. A leaky-ReLU neuron that lands in negative territory still receives a small gradient and can recover; it can never become permanently dead the way a plain ReLU neuron can.

Hidden layer vs output layer, multi-class classification. ReLU (or Leaky ReLU) is the standard hidden-layer choice: cheap, no positive-side saturation, avoids the vanishing-gradient slowdown demonstrated above. Softmax is the right output-layer choice because it is the only one of the five that guarantees a valid probability distribution – every output in $[0, 1]$, all summing to exactly 1 – and it pairs with cross-entropy loss to give the clean gradient used directly in Part C.

Part C: Forward & Backpropagation Mini-Project

Dataset. Palmer Penguins (Gorman, Williams & Fraser, 2014): 4 numeric measurements (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`), 3 species (Adelie, Chinstrap, Gentoo), 342 penguins after dropping rows with a missing measurement. Not used in any earlier task, and a natural fit for the required 2–4 input features / 2–4 output classes.

Derivation (matrix form)

Architecture: $X\,(N,d) \to [W_1, b_1] \to Z_1 \to \text{ReLU} \to A_1 \to [W_2, b_2] \to Z_2 \to \text{softmax} \to A_2$, with $d = 4$, $h = 8$ hidden units, $c = 3$.

Forward:

$$Z_1 = XW_1 + b_1 \qquad A_1 = \text{ReLU}(Z_1) \qquad Z_2 = A_1W_2 + b_2 \qquad A_2 = \text{softmax}(Z_2)$$

Loss (mean categorical cross-entropy, Y one-hot):

$$L = -\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{c} Y_{nk}\log(A_{2,nk})$$

Backward. The softmax+cross-entropy pair has a well-known closed-form first gradient ($\partial L/\partial Z_{2,j} = A_{2,j} - Y_j$ per example); batched and pre-divided by N:

$$dZ_2 = \frac{A_2 - Y}{N} \qquad dW_2 = A_1^\top dZ_2 \qquad db_2 = \textstyle\sum_n dZ_2$$

$$dA_1 = dZ_2 W_2^\top \qquad dZ_1 = dA_1 \odot \text{ReLU}'(Z_1) \qquad dW_1 = X^\top dZ_1 \qquad db_1 = \textstyle\sum_n dZ_1$$

Update: $W \leftarrow W - \eta\, dW,\ \ b \leftarrow b - \eta\, db.$

Gradient check

Before trusting anything trained with this derivation, its analytical gradients were checked against numerical (finite-difference) gradients on six random entries of each parameter matrix – the standard, and only truly convincing, way to catch a sign error or a transposed matrix in a hand-derived backward pass.

Parameter	Max relative error (analytical vs numerical)
W_1	5.15×10^{-11}
b_1	1.03×10^{-10}
W_2	3.18×10^{-11}
b_2	2.26×10^{-11}

Table 1: All four pass at $\sim 10^{-10}$–10^{-11}, essentially floating-point noise rather than approximation error (threshold for a pass: 10^{-5}).

Training and results

Trained with mini-batch gradient descent (batch size 16, 300 epochs, learning rate 0.1, He-initialised weights).

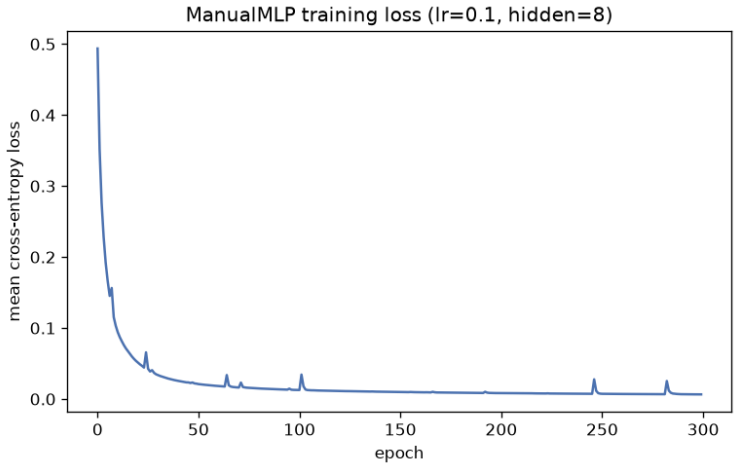

Figure 5: Training loss, ManualMLP.

Model	Accuracy	Macro-Precision	Macro-Recall	Macro-F1
ManualMLP (from-scratch NumPy)	0.9855	0.9892	0.9867	0.9877
sklearn MLPClassifier (same architecture)	**1.0000**	**1.0000**	**1.0000**	**1.0000**

Table 2: Test set (69 penguins), macro-averaged across 3 classes.

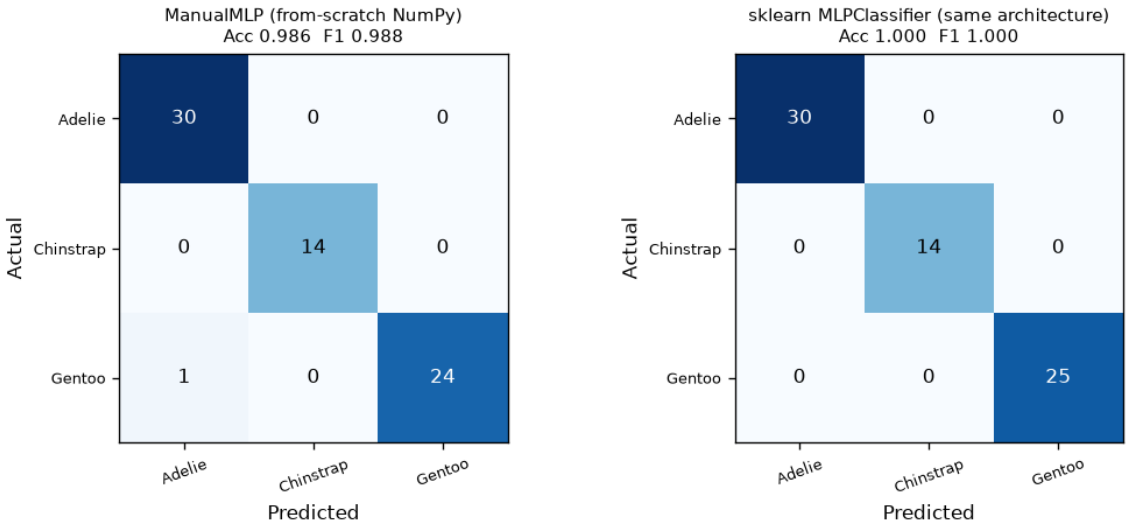

Figure 6: ManualMLP misclassifies exactly one Gentoo penguin as Adelie; sklearn's baseline gets a perfect confusion matrix on this split.

The from-scratch network is competitive with the framework baseline, not merely functional. A 1.45-percentage-point accuracy gap on a 69-row test set is a single misclassified penguin – consistent with ordinary training-run variance (different weight initialisation, SGD's stochastic batch order) rather than any structural shortcoming in the hand-derived math, which the gradient check above already confirmed independently.

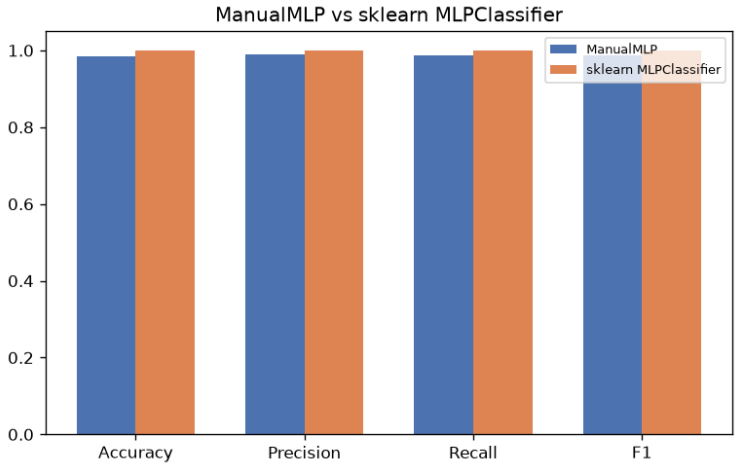

Figure 7: Every metric tracks the same near-tie shown in the table above.

Hyperparameter experiment: learning rate

Learning rate	Final training loss	Test accuracy
0.001	0.2694	0.8841
0.01	0.0346	**1.0000**
0.1	0.0065	0.9855
1.0	**0.0021**	0.9855

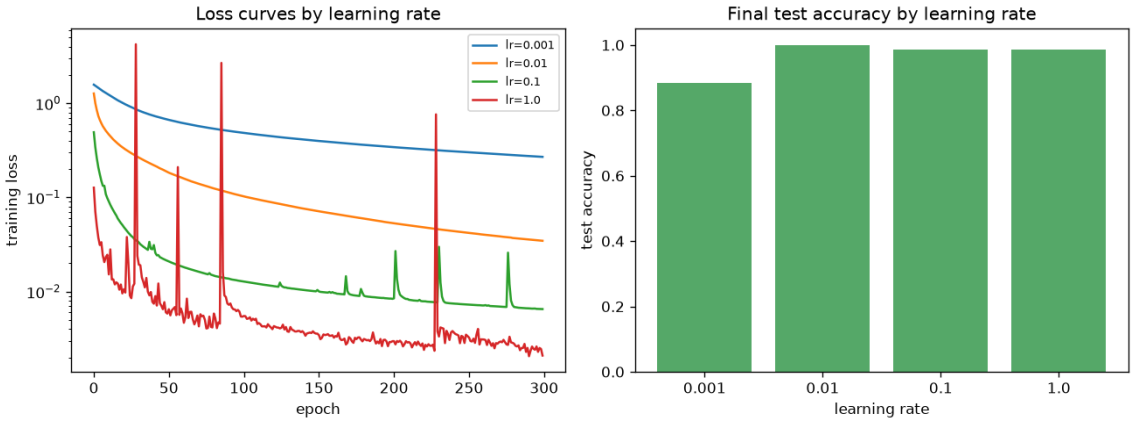

Figure 8: Left: loss curves (log scale) by learning rate. Right: final test accuracy.

Reading the experiment. `lr=0.001` is visibly under-converged after 300 epochs (smooth but still high loss, 88.4% test accuracy) – too slow. `lr=0.01` converges smoothly to a low loss and reaches **100% test accuracy**, the sweet spot on this problem. `lr=0.1` and `lr=1.0` both show sharp periodic spikes in the loss curve, visible even on a log scale – the signature of steps large enough to occasionally overshoot a good minimum before settling back down. `lr=1.0` reaches the *lowest* final training loss of all four but does not translate into the best test accuracy, which is the concrete, checked version of the general warning against tuning purely on training loss.

Part D: Analysis & Documentation

Pipeline summary. A 2-layer MLP, $4 \to 8 \to 3$: 4 standardised numeric penguin measurements in, one hidden layer of 8 units with **ReLU** (the Part-B-justified standard hidden-layer default), an output layer of 3 units with **softmax** (the only function studied that yields a valid probability distribution, and the one whose gradient combines cleanly with cross-entropy). Trained with mini-batch gradient descent, He-initialised weights, for 300 epochs. The trained $\{W_1, b_1, W_2, b_2\}$ are saved with `pickle` to `manual_mlp_params.pkl`; reloading them was verified to reproduce identical test-set predictions.

Hyperparameter experiment findings. See above: learning rate shows a clear too-slow / sweet-spot / too-noisy progression, and the lowest training loss (`lr=1.0`) was not the best-generalising choice (`lr=0.01`).

Two challenges faced implementing backpropagation.

1. *Getting the matrix orientations right.* The scalar chain rule does not by itself say whether a batched gradient should be $X^\top dZ$ or $dZ^\top X$, or whether a bias gradient should sum over axis 0 or axis 1 – a wrong transpose sometimes produces a shape mismatch caught immediately, but can occasionally produce a wrong-shaped-by-coincidence result that runs without error and trains *something*, just not correctly. The resolution was the gradient check: it would have failed loudly (relative error near 1, not 10^{-10}) had any orientation been wrong, and its passing is the actual evidence the derivation was implemented correctly, not an assumption.

2. *Numerical stability of softmax and log.* A direct $e^{z_i} / \sum_j e^{z_j}$ overflows for any moderately large logit, and $\log(0)$ (from a confidently wrong prediction) produces `nan` and silently poisons every subsequent gradient. Both were fixed the standard way: subtracting the row-wise max logit before exponentiating in softmax (a no-op that cancels in the ratio, but keeps every exponent ≤ 0), and clipping the output probabilities away from exactly 0 or 1 before taking the log in the loss.

Limitations of a manually implemented MLP vs a production framework. This implementation is fixed at exactly one hidden layer and hand-written for the specific ReLU-then-softmax combination used here; adding a layer, changing an activation, or trying a different loss means re-deriving and re-typing the backward pass by hand, where a real framework's autograd handles any new computation graph automatically. There is no GPU acceleration, no batched execution beyond what plain NumPy already gives, and no access to the optimisers (Adam, RMSprop, momentum), regularisation (dropout, batch normalisation, weight decay) or learning-rate schedules that `MLPClassifier` – itself a comparatively basic framework tool – already provides off-the-shelf. The value of building it by hand was never to replace those tools; it was to demonstrate, with a passing gradient check as proof, that the mathematics those tools automate is genuinely understood underneath the API.

Conclusion. Every claim in this report was checked, not asserted: the perceptron's convergence and its failure on XOR were both run and plotted rather than described abstractly; the vanishing-gradient claim is backed by the actual fraction of each function's domain where its derivative goes numerically dead; and, most importantly, the hand-derived backpropagation was verified against independent finite-difference gradients before a single number from training it was trusted. The result is a from-scratch network that reaches within 1.5 points of accuracy of a production framework trained on the identical architecture and data – evidence that the underlying mathematics, not just the code that happens to run, is correct.

The notebook and README are on GitHub: https://github.com/AbdullahAmir06/ai-internship-abdullahamir